Shared by Shri. This is a copy of a chat between Claude and Shri. Content may include unverified or unsafe content that does not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

Report

Files hidden when shared

I'm doing a lab where I fix and extend a small terminal checkers game in Python (standard library only, 4 files: main.py, game.py, board.py, rules.py). The README tasks and all the code are pasted below.

Do NOT change or write any code yet. Please:
1. Briefly explain what each file does and how one turn flows from input to board update.
2. Trace exactly what happens to the board when Red at (4,1) legally captures a Black piece at (3,2) by jumping to (2,3).
3. List every bug or missing rule you find against the README tasks (capture removal, win/no-move detection, forced capture, multi-capture, king movement in both directions, promotion, move feedback). Also check: can a player jump over their own king ("RK"/"BK")? Does the board print aligned when a king is on it?
4. Suggest the smallest plan to fix everything while keeping the same four files and adding no dependencies.

Updated memory

1. What each file does and how a turn flows

- **main.py**: entry point. It builds a `Checkers` object and calls `run()`.
- **board.py**: owns the `SIZE = 8` grid. `initial_board()` puts Black ("B") on rows 0–2 and Red ("R") on rows 5–7, on squares where `(r+c)` is odd. `move_piece()` copies the piece to the destination and blanks the start.
- **rules.py**: pure predicates. `simple_move` checks a one-step diagonal in the player's forward direction onto an empty square. `capture_move` checks a two-step diagonal onto an empty square with a non-empty, non-own piece in the middle. `promote` scans the back rows and turns "R" on row 0 into "RK" and "B" on row 7 into "BK".
- **game.py**: the loop. It prints the board, reads `sr sc er ec`, and handles `q`. It rejects input that isn't four numbers, is off the board, or starts on a square the player doesn't own. It then tries `capture_move`, then `simple_move`, and on success calls `move_piece`, then `promote`, then swaps the player.

One turn: print board → parse and validate input → ownership check → capture check, else simple-move check → `move_piece` → `promote` → switch player.

2. Trace: Red (4,1) captures Black (3,2) landing on (2,3)

This assumes (2,3) is empty. In the starting layout it holds a "B", so some earlier moves would have cleared it.

1. `game.py` parses `4 1 2 3`. All values are in range and `board[4][1] == "R"`, so the ownership check passes.
2. `capture_move`: direction is -1 and `er - sr = -2 = 2*direction`. The midpoint is `((4+2)//2, (1+3)//2) = (3,2)`, which holds "B", so it returns True.
3. `move_piece` sets `board[2][3] = "R"` and `board[4][1] = "."`. **Nothing touches (3,2).**
4. `promote` does nothing, and the turn passes to Black.

The result is R on (2,3), (4,1) empty, and the **Black piece still sitting on (3,2)**. That is the Task 1 defect: the jumped piece is never removed.

For your "before" video, this sequence reproduces it from the starting position: `5 2 4 3`, then `2 1 3 2`, then `4 3 2 1`. After the last move, R is on (2,1) and B is still on (3,2).

3. Bugs and missing rules

Task 1: capture removal

- The jumped piece is never removed. `move_piece` only handles start and end, and nothing else clears the midpoint.

Task 2: win and no-move detection

- Neither exists. The loop only exits on `q`. A side with no pieces or no moves just gets prompted forever.

Task 3: complete rules

- **Forced capture is missing.** A player can make a simple move even when a capture is available. Nothing scans the board for captures.
- **Multi-capture is missing.** One capture ends the turn, even if the same piece could jump again.
- **Kings only move forward.** Both predicates take direction from `player` and never look at whether the piece is a king. A king on the back row can't move at all, because its only legal direction is off the board.
- **Players can jump their own kings.** The midpoint test is `not in (".", player)`. For Red, the middle piece "RK" is not in `(".", "R")`, so it counts as an enemy. The same holds for "BK" and Black. So yes, a player can capture their own king.
- **Promotion rows are correct.** Red promotes on row 0 and Black on row 7. The promotion itself is fine, but it is currently pointless because kings can't move backward. One rule to decide: standard rules end a turn when a man is promoted mid-capture.

Task 4: feedback

- Nothing is printed for accepted moves. Only rejections produce messages.

Display

- **Misaligned with kings.** Cells are joined with a single space, and "RK"/"BK" are two characters wide while the header and other cells are one. Every cell after a king in that row shifts right by one.

Smaller things

- `EOFError` (Ctrl-D) and `KeyboardInterrupt` are unhandled, so they crash with a traceback instead of quitting cleanly.
- `SIZE` is defined in both `board.py` and `rules.py`.

4. Smallest plan (same four files, no dependencies)

rules.py

1. Add two tiny helpers: one for "does this piece belong to player" (first character) and one for "is it a king".
2. Make `simple_move` and `capture_move` read the piece at `start` and allow both directions for kings and forward only for men.
3. Fix the midpoint test to "non-empty and not owned by player." This fixes the own-king jump.
4. Add `captures_from(board, player, pos)`, `any_capture(board, player)` and `has_legal_move(board, player)`, built on the two predicates. Keep every predicate free of printing, so repeated internal checks never create duplicate output.
5. Let `promote` report which squares were promoted, so `game.py` can mention it.

board.py

6. Make `move_piece` remove the jumped piece when the move is a 2-step jump, and return what it captured. This is the Task 1 fix.
7. Optionally add a small piece-count helper.

game.py

8. After a move, check the *next* player: no pieces or no legal move means print the winner and return.
9. Forced capture: if `any_capture` is true and the attempted move isn't a capture, reject it with a message.
10. Multi-capture: after a capture, if the moving piece wasn't just promoted and `captures_from(end)` is non-empty, keep the same player. Store the locked square (for example `self.chain`) and require the next input to start there and be a capture.
11. Feedback: build one result string per accepted move (move, capture, promotion) and print it once. A multi-jump then prints one line per jump. If the README's "one action result" is meant to cover a whole chain, ask your instructor, because that changes how input works.
12. Print each cell padded to width 2, with the header padded to match.
13. Optionally catch `EOFError` and `KeyboardInterrupt` and treat them as quit.

Suggested order: record the "before" video, then fix capture removal (6) and re-run the repro. Then do the rules refactor (1–4) and promotion (5), then forced capture and multi-capture (9–

10), then end detection (8), then feedback and alignment (11–12).

Testing: assigning `Checkers().board` directly lets you set up positions for captures, double jumps, kings, no-move cases and so on. A throwaway script, or `unittest` from the standard library, would cover that without adding dependencies.

> Now fix Task 1 only: after a legal capture, the jumped opponent piece must be removed from the board, and the moving piece must still end on its destination.
>
> Requirements:
> - Smallest possible change; keep existing function names and the file structure.
> - Also fix capture_move so a player cannot jump over their own king (compare the first letter of the piece, so "RK" counts as Red and "BK" as Black).
> - Functions in rules.py must stay pure: no print() or input() inside them.
> - Do not add any other features yet.
>
> Give me the complete updated contents of every file you change (full files, not diffs), then the exact inputs to type to verify the fix starting from the initial board, and what the board should look like afterwards.

Two files change: `board.py` and `rules.py`. `game.py` stays as is.

The removal goes in `move_piece`, because it is the only place that mutates the board. A 2-row move can only reach it after `capture_move` has validated it, so the midpoint check is safe. `simple_move` always moves one row, so it never triggers removal.

board.py

python

```python
SIZE = 8


def initial_board():
    board = [["."] * SIZE for _ in range(SIZE)]
    for r in range(3):
        for c in range(SIZE):
            if (r + c) % 2 == 1:
                board[r][c] = "B"
    for r in range(5, 8):
        for c in range(SIZE):
            if (r + c) % 2 == 1:
                board[r][c] = "R"
    return board


def move_piece(board, start, end):
    board[end[0]][end[1]] = board[start[0]][start[1]]
    board[start[0]][start[1]] = "."
    # A two-row move is a jump: clear the square that was jumped over.
    if abs(end[0] - start[0]) == 2:
        board[(start[0] + end[0]) // 2][(start[1] + end[1]) // 2] = "."
```

rules.py

python

```
SIZE = 8


def simple_move(board, player, start, end):
    sr, sc = start
    er, ec = end
    direction = -1 if player == "R" else 1
    return (
        board[er][ec] == "." and
        abs(er - sr) == 1 and abs(ec - sc) == 1 and
        er - sr == direction
    )


def capture_move(board, player, start, end):
    sr, sc = start
    er, ec = end
    direction = -1 if player == "R" else 1
    mr, mc = (sr + er) // 2, (sc + ec) // 2
    return (
        board[er][ec] == "." and
        abs(er - sr) == 2 and abs(ec - sc) == 2 and
        er - sr == 2 * direction and
        board[mr][mc] != "." and board[mr][mc][0] != player
    )


def promote(board):
    for c in range(SIZE):
        if board[0][c] == "R":
            board[0][c] = "RK"
        if board[SIZE - 1][c] == "B":
            board[SIZE - 1][c] = "BK"
```

The only change in `capture_move` is the last condition. The middle square must be non-empty and its first letter must differ from `player`, so "RK" counts as Red and "BK" as Black.

Verify from the initial board

Run `python main.py` and type these three moves:

1. `5 2 4 3`: Red moves forward.
2. `2 1 3 2`: Black moves forward, next to Red.
3. `4 3 2 1`: Red jumps over (3,2) and lands on (2,1).

After the third move, the board should be:

```
   0 1 2 3 4 5 6 7
0  . B . B . B . B
1  B . B . B . B .
2  . R . B . B . B
3  . . . . . . . .
4  . . . . . . . .
5  R . . . R . R .
6  . R . R . R . R
7  R . R . R . R .
```

Check these three things. The Black piece on (3,2) is gone, where it used to remain before the fix. R is on (2,1) and (4,3) is empty. The prompt now says `B>`.

To confirm it can't be captured again, try `1 0 3 2` as Black from this position. It should print "Invalid move." because (2,1) is Red and (3,2) is empty, so there's nothing to jump.

Verify the own-king fix

This needs a custom position, so use a Python REPL in the project folder:

python

```python
from rules import capture_move
b = [["."] * 8 for _ in range(8)]
b[4][1] = "R"
b[3][2] = "RK"
print(capture_move(b, "R", (4, 1), (2, 3)))  # expect False (own king)
b[3][2] = "BK"
print(capture_move(b, "R", (4, 1), (2, 3)))  # expect True (enemy king)
```

I haven't run any of this here. Please run it yourself before recording the "after" video.

Now implement Tasks 2 and 3 together, since both depend on knowing a player's legal moves. Keep the same four files, standard library only, all state in memory.

Implement exactly these rules:
- Pieces: "R"/"B" are men, "RK"/"BK" are kings. Red moves toward row 0, Black toward row 7. Men move and capture forward only. Kings move one step or jump one enemy piece in all four diagonal directions.
- Forced capture: if the current player has any capture available with any piece, a non-capturing move is rejected with a clear message.
- Multi-capture: after a capture, if the SAME piece can capture again from its landing square, the turn does not pass. The same player must continue, and only that piece may be moved. Each jump is typed as its own "sr sc er ec" input. The chain ends when no further capture is available.
- Promotion: a Red man reaching row 0 becomes "RK"; a Black man reaching row 7 becomes "BK". If a man is promoted in the middle of a capture chain, the chain ends and the turn passes.
- End of game: before each turn, if the player to move has no pieces or no legal moves, print who won and why, then exit run() cleanly.

- Put the logic in rules.py as pure functions (no print/input), e.g. functions returning a piece's captures, all captures for a player, and whether a player has any legal move. game.py handles only input, printing and turn flow.
- Fix print_board so columns stay aligned when a king is on the board, without changing how pieces are stored.
- Keep invalid-coordinate handling and "q" to quit working.

Give me the complete updated contents of every file you change, then the exact inputs I can type to see forced capture being enforced, starting from the initial board.

Show more

Only `rules.py` and `game.py` change. `board.py` (with the jump-removal from Task 1) and `main.py` stay exactly as they are. I haven't run this code here, so please run the checks below before recording your "after" video.

Two small side effects to know about:

- `promote()` now returns `True` if it promoted anything. Only the piece that just moved can be promoted, so `game.py` can use that to end a capture chain.
- During a chain the prompt reads `R (continue from 4 3)>`, so you can tell the turn didn't pass. That is only an input hint, not the Task 4 move feedback.

rules.py

python

```
SIZE = 8

KING_DIRS = [(-1, -1), (-1, 1), (1, -1), (1, 1)]
MAN_DIRS = {"R": [(-1, -1), (-1, 1)], "B": [(1, -1), (1, 1)]}


def in_bounds(pos):
    return 0 <= pos[0] < SIZE and 0 <= pos[1] < SIZE


def owns(player, piece):
    """True if the piece ("R", "RK", "B", "BK") belongs to player."""
    return piece != "." and piece[0] == player


def piece_directions(piece):
    """(row, col) steps this piece may use. Kings: all four; men: forward only."""
    if piece == ".":
        return []
    if len(piece) == 2:
        return KING_DIRS
    return MAN_DIRS[piece]


def simple_move(board, player, start, end):
    if not (in_bounds(start) and in_bounds(end)):
        return False
    sr, sc = start
    er, ec = end
    piece = board[sr][sc]
    return (
        owns(player, piece) and
        board[er][ec] == "." and
        (er - sr, ec - sc) in piece_directions(piece)
    )


def capture_move(board, player, start, end):
    if not (in_bounds(start) and in_bounds(end)):
        return False
    sr, sc = start
    er, ec = end
    piece = board[sr][sc]
    if not owns(player, piece) or board[er][ec] != ".":
        return False
    jumps = [(2 * dr, 2 * dc) for dr, dc in piece_directions(piece)]
    if (er - sr, ec - sc) not in jumps:
        return False
```

```
    middle = board[(sr + er) // 2][(sc + ec) // 2]
    return middle != "." and not owns(player, middle)


def pieces_of(board, player):
    return [
        (r, c)
        for r in range(SIZE)
        for c in range(SIZE)
        if owns(player, board[r][c])
    ]


def captures_from(board, player, pos):
    """Landing squares of every capture available to the piece at pos."""
    r, c = pos
    ends = []
    for dr, dc in piece_directions(board[r][c]):
        end = (r + 2 * dr, c + 2 * dc)
        if capture_move(board, player, pos, end):
            ends.append(end)
    return ends


def all_captures(board, player):
    """List of (start, end) for every capture any of player's pieces can make."""
    result = []
    for pos in pieces_of(board, player):
        for end in captures_from(board, player, pos):
            result.append((pos, end))
    return result


def has_legal_move(board, player):
    if all_captures(board, player):
        return True
    for r, c in pieces_of(board, player):
        for dr, dc in piece_directions(board[r][c]):
            if simple_move(board, player, (r, c), (r + dr, c + dc)):
                return True
    return False


def promote(board):
    """Promote men on their back row. Returns True if a promotion happened."""
    promoted = False
    for c in range(SIZE):
        if board[0][c] == "R":
```

```
            board[0][c] = "RK"
            promoted = True
        if board[SIZE - 1][c] == "B":
            board[SIZE - 1][c] = "BK"
            promoted = True
    return promoted
```

game.py

python

```
from board import initial_board, move_piece, SIZE
from rules import (
    simple_move, capture_move, promote,
    captures_from, all_captures, has_legal_move, pieces_of,
)

NAMES = {"R": "Red", "B": "Black"}


class Checkers:
    def __init__(self):
        self.board = initial_board()
        self.player = "R"
        self.chain = None  # square of the piece that must keep capturing

    def print_board(self):
        print("\n   " + " ".join(f"{c:<2}" for c in range(SIZE)).rstrip())
        for r, row in enumerate(self.board):
            print(f"{r}  " + " ".join(f"{cell:<2}" for cell in row).rstrip())

    def game_over(self):
        opponent = "B" if self.player == "R" else "R"
        if not pieces_of(self.board, self.player):
            print(f"{NAMES[opponent]} wins: {NAMES[self.player]} has no pieces left.")
            return True
        if not has_legal_move(self.board, self.player):
            print(f"{NAMES[opponent]} wins: {NAMES[self.player]} has no legal moves.")
            return True
        return False

    def run(self):
        print("Checkers — move: sr sc er ec")
        while True:
            self.print_board()
            if self.game_over():
                return

            prompt = f"{self.player}> "
            if self.chain is not None:
                prompt = f"{self.player} (continue from {self.chain[0]} {self.chain[1]}
            raw = input(prompt).strip().lower().split()
            if raw == ["q"]:
                return
            if len(raw) != 4:
                print("Enter four coordinates.")
                continue
            try:
                sr, sc, er, ec = map(int, raw)
```

```
            except ValueError:
                print("Coordinates must be numbers.")
                continue
            if not all(0 <= x < SIZE for x in (sr, sc, er, ec)):
                print("Outside board.")
                continue
            if self.board[sr][sc] not in (self.player, self.player + "K"):
                print("That is not your piece.")
                continue

            start, end = (sr, sc), (er, ec)

            if self.chain is not None and start != self.chain:
                print(f"You must keep capturing with the piece at {self.chain[0]} {self
                continue

            if capture_move(self.board, self.player, start, end):
                captured = True
            elif self.chain is not None:
                print("Invalid capture. You must continue capturing with that piece.")
                continue
            elif simple_move(self.board, self.player, start, end):
                options = all_captures(self.board, self.player)
                if options:
                    listing = ", ".join(
                        f"{s[0]} {s[1]} -> {e[0]} {e[1]}" for s, e in options
                    )
                    print(f"Capture is mandatory. Available: {listing}")
                    continue
                captured = False
            else:
                print("Invalid move.")
                continue

            move_piece(self.board, start, end)
            promoted = promote(self.board)

            if captured and not promoted and captures_from(self.board, self.player, enc
                self.chain = end  # same piece must jump again; turn does not pass
            else:
                self.chain = None
                self.player = "B" if self.player == "R" else "R"
```

Verify forced capture from the initial board

Run `python main.py` and type:

1. `5 2 4 3`: Red moves.

2. `2 1 3 2`: Black moves next to Red. Red now has a capture available.
3. `5 0 4 1`: a plain Red move. It should be rejected with `Capture is mandatory. Available: 4 3 -> 2 1`. The board stays the same and the prompt is still `R>`.
4. `4 3 2 1`: the capture is accepted. The Black piece on (3,2) disappears. Red can't jump again from (2,1), so the turn passes to Black.
5. `2 3 3 4`: a plain Black move. It should be rejected with `Capture is mandatory. Available: 1 0 -> 3 2, 1 2 -> 3 0`.
6. `1 0 3 2`: the capture is accepted and the Red piece on (2,1) is removed.

Steps 3 and 5 are the forced-capture checks. They show the move is rejected, the message names the legal captures, and the same player is still prompted.

Verify multi-capture and end of game

This needs a custom position, so use a REPL in the project folder:

```python
from game import Checkers
g = Checkers()
g.board = [["."] * 8 for _ in range(8)]
g.board[6][1] = "R"; g.board[7][6] = "R"
g.board[5][2] = "B"; g.board[3][4] = "B"
g.run()
```

Then type:

1. `6 1 4 3`: the first jump. The prompt changes to `R (continue from 4 3)>`.
2. `7 6 6 5`: rejected with a message saying you must keep capturing with the piece at 4 3.
3. `4 3 2 5`: the second jump. Black has no pieces left, so it prints `Red wins: Black has no pieces left.` and exits.

I traced these sequences by hand against the code, and the messages above are what the trace produces.

Task 4: every accepted move must produce exactly ONE short player-facing result line, built in game.py after the move is accepted. For example:
- "R moved (5,0) -> (4,1)"
- "R captured B at (3,2): (4,1) -> (2,3)"
- a promotion line when a man becomes a king
- during a multi-capture chain, each jump prints its own capture line, followed by a line saying which piece must continue.
Rejected moves print one error line and do not count as moves. Functions in rules.py must never print, so calling them several times (forced-capture checks, legal-move checks) can never produce duplicate messages.

Also add tests.py using only the standard library (unittest) that builds boards directly and checks: ordinary move, capture removes the jumped piece and it

cannot be captured again, own king cannot be jumped, forced capture, multi-capture, promotion on the correct back row for both colours, king moving backward, a position where a player has no legal moves, a player with no pieces, invalid coordinates, quitting with "q", and that one accepted move prints exactly one result line (use unittest.mock.patch on builtins.input and capture stdout).

Give me the complete updated game.py (and any other changed file) and the full tests.py, plus the command to run the tests.

Show more

I'll build the project in a scratch folder and run the tests before giving you the files.

Created 5 files, ran a command

All 18 tests pass. I'm now playing the real game through `main.py` with piped input as a last check, then I'll send you the files.

Ran a command

I ran the full test suite and a scripted game through `main.py`. All 18 tests pass. The scripted game gave one result line per accepted move and one error line for the rejected plain move.

Only `game.py` changes in the project, plus the new `tests.py`. `board.py`, `rules.py` and `main.py` stay as in the last round. Your spec seemed to contradict itself in two places, so I decided as follows:

- **Promotion:** you asked for one line per accepted move and also for a promotion line. I put the promotion into the move line, for example `R moved (1,2) -> (0,1), promoted to king`.
- **Chains:** each jump prints its capture line and then a second line saying which piece must continue. That is the only case where one accepted move prints two lines.

I also changed how promotion is detected. `game.py` now checks whether the piece that just moved turned from a man into a king, instead of using `promote()`'s return value.

game.py

python

```
from board import initial_board, move_piece, SIZE
from rules import (
    simple_move, capture_move, promote,
    captures_from, all_captures, has_legal_move, pieces_of,
)

NAMES = {"R": "Red", "B": "Black"}


class Checkers:
    def __init__(self):
        self.board = initial_board()
        self.player = "R"
        self.chain = None  # square of the piece that must keep capturing

    def print_board(self):
        print("\n   " + " ".join(f"{c:<2}" for c in range(SIZE)).rstrip())
        for r, row in enumerate(self.board):
            print(f"{r}  " + " ".join(f"{cell:<2}" for cell in row).rstrip())

    def game_over(self):
        opponent = "B" if self.player == "R" else "R"
        if not pieces_of(self.board, self.player):
            print(f"{NAMES[opponent]} wins: {NAMES[self.player]} has no pieces left.")
            return True
        if not has_legal_move(self.board, self.player):
            print(f"{NAMES[opponent]} wins: {NAMES[self.player]} has no legal moves.")
            return True
        return False

    def run(self):
        print("Checkers — move: sr sc er ec")
        while True:
            self.print_board()
            if self.game_over():
                return

            prompt = f"{self.player}> "
            if self.chain is not None:
                prompt = f"{self.player} (continue from {self.chain[0]} {self.chain[1]}"
            raw = input(prompt).strip().lower().split()
            if raw == ["q"]:
                return
            if len(raw) != 4:
                print("Enter four coordinates.")
                continue
            try:
                sr, sc, er, ec = map(int, raw)
```

```
            except ValueError:
                print("Coordinates must be numbers.")
                continue
            if not all(0 <= x < SIZE for x in (sr, sc, er, ec)):
                print("Outside board.")
                continue
            if self.board[sr][sc] not in (self.player, self.player + "K"):
                print("That is not your piece.")
                continue

            start, end = (sr, sc), (er, ec)

            if self.chain is not None and start != self.chain:
                print(f"You must keep capturing with the piece at ({self.chain[0]},{sel
                continue

            if capture_move(self.board, self.player, start, end):
                captured = True
            elif self.chain is not None:
                print("Invalid capture. You must continue capturing with that piece.")
                continue
            elif simple_move(self.board, self.player, start, end):
                options = all_captures(self.board, self.player)
                if options:
                    listing = ", ".join(
                        f"{s[0]} {s[1]} -> {e[0]} {e[1]}" for s, e in options
                    )
                    print(f"Capture is mandatory. Available: {listing}")
                    continue
                captured = False
            else:
                print("Invalid move.")
                continue

            # The move is accepted. Gather what the result line needs
            # before the board changes.
            was_man = len(self.board[sr][sc]) == 1
            if captured:
                mr, mc = (sr + er) // 2, (sc + ec) // 2
                victim = self.board[mr][mc]

            move_piece(self.board, start, end)
            promote(self.board)
            promoted = was_man and len(self.board[er][ec]) == 2

            # Exactly one result line per accepted move.
            if captured:
                result = (f"{self.player} captured {victim} at ({mr},{mc}): "
```

```
                    f"({sr},{sc}) -> ({er},{ec})")
            else:
                result = f"{self.player} moved ({sr},{sc}) -> ({er},{ec})"
            if promoted:
                result += ", promoted to king"
            print(result)

            if captured and not promoted and captures_from(self.board, self.player, enc
                self.chain = end  # same piece must jump again; turn does not pass
                print(f"{self.player} must continue capturing with the piece at ({er},{
            else:
                self.chain = None
                self.player = "B" if self.player == "R" else "R"
```

tests.py

python

```
import io
import re
import unittest
from contextlib import redirect_stdout
from unittest.mock import patch

from board import SIZE
from game import Checkers
from rules import (
    all_captures, capture_move, captures_from, has_legal_move,
    pieces_of, promote, simple_move,
)


def make_game(pieces, player="R"):
    """Build a Checkers game on an empty board with the given pieces."""
    g = Checkers()
    g.board = [["."] * SIZE for _ in range(SIZE)]
    for (r, c), piece in pieces.items():
        g.board[r][c] = piece
    g.player = player
    g.chain = None
    return g


def play(game, inputs):
    """Run the game with scripted input; a final 'q' guarantees it stops."""
    out = io.StringIO()
    with patch("builtins.input", side_effect=list(inputs) + ["q"]), redirect_stdout(out
        game.run()
    return out.getvalue()


BOARD_ROW = re.compile(r"^\d  ")


def messages(output):
    """Output lines that are neither board drawing nor the banner."""
    lines = []
    for line in output.splitlines():
        if not line.strip():
            continue
        if line.startswith("   ") or BOARD_ROW.match(line):
            continue
        if line.startswith("Checkers"):
            continue
        lines.append(line)
    return lines
```

```
class TestMoves(unittest.TestCase):
    def test_ordinary_move(self):
        g = make_game({(5, 0): "R", (0, 7): "B"})
        out = play(g, ["5 0 4 1"])
        self.assertEqual(g.board[4][1], "R")
        self.assertEqual(g.board[5][0], ".")
        self.assertEqual(g.player, "B")
        self.assertEqual(messages(out), ["R moved (5,0) -> (4,1)"])

    def test_king_moves_backward(self):
        g = make_game({(3, 2): "RK", (0, 1): "B"})
        out = play(g, ["3 2 4 3"])
        self.assertEqual(g.board[4][3], "RK")
        self.assertEqual(g.board[3][2], ".")
        self.assertEqual(messages(out), ["R moved (3,2) -> (4,3)"])

        h = make_game({(4, 3): "BK", (6, 5): "R"}, player="B")
        out = play(h, ["4 3 3 2"])
        self.assertEqual(h.board[3][2], "BK")
        self.assertEqual(messages(out), ["B moved (4,3) -> (3,2)"])

    def test_king_captures_backward(self):
        g = make_game({(3, 2): "RK", (4, 3): "B", (0, 7): "B"})
        out = play(g, ["3 2 5 4"])
        self.assertEqual(g.board[4][3], ".")
        self.assertEqual(g.board[5][4], "RK")
        self.assertEqual(messages(out), ["R captured B at (4,3): (3,2) -> (5,4)"])


class TestCaptures(unittest.TestCase):
    def test_capture_removes_jumped_piece_and_it_cannot_be_captured_again(self):
        g = make_game({(4, 1): "R", (4, 3): "R", (3, 2): "B", (0, 7): "B"})
        out = play(g, ["4 1 2 3"])
        self.assertEqual(g.board[3][2], ".")
        self.assertEqual(g.board[2][3], "R")
        self.assertEqual(g.board[4][1], ".")
        self.assertEqual(messages(out), ["R captured B at (3,2): (4,1) -> (2,3)"])
        # The other Red piece can no longer jump the removed piece.
        self.assertFalse(capture_move(g.board, "R", (4, 3), (2, 1)))
        self.assertNotIn(((4, 3), (2, 1)), all_captures(g.board, "R"))

    def test_own_king_cannot_be_jumped(self):
        g = make_game({(4, 1): "R", (3, 2): "RK", (0, 7): "B"})
        self.assertFalse(capture_move(g.board, "R", (4, 1), (2, 3)))
        out = play(g, ["4 1 2 3"])
        self.assertEqual(messages(out), ["Invalid move."])
```

```
        self.assertEqual(g.board[3][2], "RK")
        self.assertEqual(g.board[4][1], "R")
        self.assertEqual(g.player, "R")

        h = make_game({(3, 2): "B", (4, 3): "BK"}, player="B")
        self.assertFalse(capture_move(h.board, "B", (3, 2), (5, 4)))

    def test_forced_capture(self):
        g = make_game({(4, 3): "R", (6, 1): "R", (3, 2): "B", (0, 7): "B"})
        out = play(g, ["6 1 5 0"])  # plain move while a capture exists
        msgs = messages(out)
        self.assertEqual(len(msgs), 1)
        self.assertIn("Capture is mandatory", msgs[0])
        self.assertEqual(g.board[6][1], "R")
        self.assertEqual(g.board[5][0], ".")
        self.assertEqual(g.player, "R")

        out = play(g, ["4 3 2 1"])  # the capture itself is accepted
        self.assertEqual(g.board[3][2], ".")
        self.assertEqual(g.board[2][1], "R")
        self.assertEqual(g.player, "B")
        self.assertEqual(messages(out), ["R captured B at (3,2): (4,3) -> (2,1)"])

    def test_multi_capture_same_piece_must_continue(self):
        pieces = {(6, 1): "R", (7, 6): "R", (5, 2): "B", (3, 4): "B", (0, 1): "B"}

        g1 = make_game(pieces)
        play(g1, ["6 1 4 3"])
        self.assertEqual(g1.chain, (4, 3))
        self.assertEqual(g1.player, "R")  # turn did not pass

        g = make_game(pieces)
        out = play(g, ["6 1 4 3", "7 6 6 5", "4 3 2 5"])
        self.assertEqual(messages(out), [
            "R captured B at (5,2): (6,1) -> (4,3)",
            "R must continue capturing with the piece at (4,3)",
            "You must keep capturing with the piece at (4,3).",
            "R captured B at (3,4): (4,3) -> (2,5)",
        ])
        self.assertEqual(g.board[5][2], ".")
        self.assertEqual(g.board[3][4], ".")
        self.assertEqual(g.board[2][5], "R")
        self.assertEqual(g.board[7][6], "R")  # the other piece never moved
        self.assertIsNone(g.chain)
        self.assertEqual(g.player, "B")


class TestPromotion(unittest.TestCase):
    def test_red_promotes_on_row_0(self):
```

```
        g = make_game({(1, 2): "R", (3, 0): "B"})
        out = play(g, ["1 2 0 1"])
        self.assertEqual(g.board[0][1], "RK")
        self.assertEqual(messages(out), ["R moved (1,2) -> (0,1), promoted to king"])
        self.assertEqual(g.player, "B")

    def test_black_promotes_on_row_7(self):
        g = make_game({(6, 1): "B", (3, 4): "R"}, player="B")
        out = play(g, ["6 1 7 2"])
        self.assertEqual(g.board[7][2], "BK")
        self.assertEqual(messages(out), ["B moved (6,1) -> (7,2), promoted to king"])
        self.assertEqual(g.player, "R")

    def test_men_are_not_promoted_off_the_back_row(self):
        g = make_game({(6, 1): "R", (1, 2): "B"})
        play(g, ["6 1 5 0"])
        self.assertEqual(g.board[5][0], "R")
        play(g, ["1 2 2 3"])
        self.assertEqual(g.board[2][3], "B")

    def test_promotion_mid_capture_ends_the_chain(self):
        # After landing on row 0 the new king could jump (1,6), but the turn must pass.
        g = make_game({(2, 3): "R", (1, 4): "B", (1, 6): "B"})
        out = play(g, ["2 3 0 5"])
        self.assertEqual(g.board[0][5], "RK")
        self.assertTrue(captures_from(g.board, "R", (0, 5)))
        self.assertIsNone(g.chain)
        self.assertEqual(g.player, "B")
        self.assertEqual(
            messages(out),
            ["R captured B at (1,4): (2,3) -> (0,5), promoted to king"],
        )


class TestGameEnd(unittest.TestCase):
    def test_player_with_no_legal_moves_loses(self):
        g = make_game({(1, 0): "B", (2, 1): "R", (3, 2): "R", (6, 5): "R"})
        self.assertFalse(has_legal_move(g.board, "B"))
        out = play(g, ["6 5 5 4"])
        self.assertEqual(messages(out), [
            "R moved (6,5) -> (5,4)",
            "Red wins: Black has no legal moves.",
        ])

    def test_player_with_no_pieces_loses(self):
        g = make_game({(4, 1): "R", (3, 2): "B"})
        out = play(g, ["4 1 2 3"])
        self.assertEqual(messages(out), [
            "R captured B at (3,2): (4,1) -> (2,3)",
```

```
            "Red wins: Black has no pieces left.",
        ])

        h = make_game({(4, 1): "R"}, player="B")
        self.assertEqual(messages(play(h, [])), ["Red wins: Black has no pieces left."]


class TestInputAndFeedback(unittest.TestCase):
    def test_invalid_input_is_rejected_without_changing_state(self):
        g = make_game({(5, 0): "R", (0, 7): "B"})
        before = [row[:] for row in g.board]
        out = play(g, ["abc 1 2 3", "1 2 3", "9 9 8 8", "-1 0 0 1",
                       "0 0 1 1", "5 0 3 2", ""])
        self.assertEqual(messages(out), [
            "Coordinates must be numbers.",
            "Enter four coordinates.",
            "Outside board.",
            "Outside board.",
            "That is not your piece.",
            "Invalid move.",
            "Enter four coordinates.",
        ])
        self.assertEqual(g.board, before)
        self.assertEqual(g.player, "R")
        self.assertIsNone(g.chain)

    def test_quit(self):
        g = make_game({(5, 0): "R", (0, 7): "B"})
        with patch("builtins.input", side_effect=["q"]) as mock_input, \
                redirect_stdout(io.StringIO()) as out:
            result = g.run()
        self.assertIsNone(result)
        self.assertEqual(mock_input.call_count, 1)
        self.assertEqual(messages(out.getvalue()), [])

        h = make_game({(5, 0): "R", (0, 7): "B"})
        with patch("builtins.input", side_effect=["5 0 4 1", "Q"]) as mock_input, \
                redirect_stdout(io.StringIO()):
            h.run()
        self.assertEqual(mock_input.call_count, 2)
        self.assertEqual(h.board[4][1], "R")

    def test_one_result_line_per_accepted_move(self):
        cases = [
            ({(5, 0): "R", (0, 7): "B"}, "5 0 4 1", "R"),
            ({(4, 1): "R", (3, 2): "B", (0, 7): "B"}, "4 1 2 3", "R"),
            ({(1, 2): "R", (3, 0): "B"}, "1 2 0 1", "R"),
            ({(6, 1): "B", (3, 4): "R"}, "6 1 7 2", "B"),
        ]
```

```
        for pieces, move, player in cases:
            with self.subTest(move=move):
                g = make_game(pieces, player)
                msgs = messages(play(g, [move]))
                self.assertEqual(len(msgs), 1)
                self.assertTrue(msgs[0].startswith(player + " "))

    def test_rules_functions_never_print(self):
        g = make_game({(4, 1): "R", (3, 2): "B", (0, 7): "B"})
        out = io.StringIO()
        with redirect_stdout(out):
            simple_move(g.board, "R", (4, 1), (3, 0))
            capture_move(g.board, "R", (4, 1), (2, 3))
            captures_from(g.board, "R", (4, 1))
            all_captures(g.board, "R")
            has_legal_move(g.board, "R")
            pieces_of(g.board, "R")
            promote(g.board)
        self.assertEqual(out.getvalue(), "")

    def test_board_stays_aligned_with_a_king(self):
        g = make_game({(3, 2): "RK", (4, 3): "B"})
        out = io.StringIO()
        with redirect_stdout(out):
            g.print_board()
        lines = out.getvalue().splitlines()
        header = lines[1]
        king_row = lines[1 + 1 + 3]  # header, then rows 0..7 -> row 3
        self.assertEqual(header.index("2"), king_row.index("RK"))
        row4 = lines[1 + 1 + 4]
        self.assertEqual(header.index("3"), row4.index("B"))


if __name__ == "__main__":
    unittest.main()
```

Run the tests

From the project folder:

```
python -m unittest tests -v
```

I ran this and got `Ran 18 tests ... OK`. `tests.py` is an extra file beyond your original four, but it uses only the standard library and keeps no state outside memory.

Scripted game

The scripted game through `main.py` used these inputs: `5 2 4 3`, `2 1 3 2`, `5 0 4 1`, `4 3 2 1`, then `q`. The message lines were:

```
R moved (5,2) -> (4,3)
B moved (2,1) -> (3,2)
Capture is mandatory. Available: 4 3 -> 2 1
R captured B at (3,2): (4,3) -> (2,1)
```

Each accepted move printed one result line, and the rejected plain move printed one error line and did not count as a move.